Bit-Exact by Construction: A Verification-First RTL Accelerator that Inherits the GGUF k-Quant Checkpoint Ecosystem

Wick-Lim
Independent Researcher
wicklim90@gmail.com
https://github.com/Wick-Lim/AIPU

July 22, 2026

Abstract

We present a synthesizable RTL accelerator for running a frontier open-weight mixture-of-experts model—the 753B-parameter GLM-5.2, in its published ∼467 GB GGUF k-quant form—as a single-user, fully offline appliance. The work is deliberately *verification-first* and *pre-silicon*: no fabricated chip, no measured end-to-end token rate. The central claim is a *bit-exactness contract*: the dequantization of GGUF k-quants (Q4_K/Q6_K/Q8_0) is proven, via an independently cross-checked reference, bitwise-equal to llama.cpp's own `ggml` kernels on 376,586,240 weights from two real published GGUF files, so the silicon inherits the community's checkpoint ecosystem by construction. This closes the "self-referential golden" gap endemic to reimplementation-based accelerators. Around the contract we contribute: (i) speculative decoding proven in RTL—the committed stream is a bit-exact prefix of greedy decoding (`spec==greedy`), enabled by an intra-batch causal attention mode and a die-internal KV write-back, with the amortization A_{eff} measured from a hardware weight-load counter; (ii) an adversarial verification discipline—load-bearing gates paired with must-fail fault injections, exact per-gate test counts pinned by the release gate, and the finding that *output-insensitive* inputs (MoE router codes absorbed by top-k; normalization gains) defeat end-to-end equality checks and require direct per-beat die-input bindings; and (iii) a bandwidth-bound residency design whose roofline uses measured expert-union growth and accept rates, with its stall mechanism validated on real RTL cycles (∼73× under residency). The datapath places and routes on a commodity FPGA. All projected token rates are tagged as estimates; the remaining physical gates—vendor PHY IP and tapeout—are named, not hidden.

1 Introduction

Frontier large language models are increasingly available as open weights, yet running the largest of them locally—offline, on data that must not leave the premises—remains out of reach for most users. The obstacle is not compute but memory bandwidth. A 753B-parameter MoE model quantized to roughly four bits occupies ∼467 GB; each decoded token reads on the order of 25 GB of active weights, so single-stream throughput is governed by bandwidth ÷ bytes-per-token, and arithmetic units spend most of their time waiting. Consumer multi-GPU rigs cannot hold 467 GB in aggregate VRAM, so weights spill to host memory and stream over PCIe; the effective bandwidth—not the number of GPUs—then caps throughput at a few tokens per second. Datacenter deployments that

keep the whole model resident in high-bandwidth memory reach hundreds of tokens per second, but at a cost and form factor incompatible with a single-user appliance.

This paper studies the middle of that gap: an application-specific design that holds the entire model resident in a single, wide, moderately-fast memory pool, purpose-built to saturate the memory-bound roofline rather than to provide general-purpose FLOPS. The target is a local box that runs the full model with the network disconnected—an audit as literal as "does it still work with the Ethernet cable unplugged?"—which is precisely the property air-gapped organizations require and which cloud, in-VPC, and trusted-execution deployments by construction cannot provide.

The contract. A design of this kind is only credible if its claims are checkable. Machine-learning accelerators are frequently validated against a reimplementation of the reference numerics, leaving a "self-referential golden" gap: the hardware may be bit-exact to the author's own reference while diverging from the software the community actually runs. Prior FPGA work on GGUF-format kernels [9, 10] accelerates k-quant matrix multiplies but states no bitwise-equivalence contract with `ggml`; and systems that run GGUF checkpoints on other backends via format conversion do not in general preserve bitwise numerics. We instead make the contract the headline: the RTL dequantization is bit-exact to an independent reference implementation, and that reference is proven bitwise-identical to llama.cpp's own `dequantize_row_*` kernels on every Q4_K and Q6_K super-block and every Q8_0 block of two real published GGUF files—376,586,240 weights compared as raw `uint32`, no tolerance. By transitivity, the silicon dequantizes real checkpoint bytes exactly as the software ecosystem does, so it inherits the GGUF checkpoint zoo *by construction*. We are equally explicit about what the contract excludes: whole-runtime numeric equality with llama.cpp (attention and accumulation orders differ by design) and any claim about fabricated silicon.

Contributions.

1. **A bit-exact GGUF k-quant datapath, cross-validated against llama.cpp on real checkpoint bytes** (Sections 3 and 5). The RTL dequantization of Q4_K/Q6_K/Q8_0/F16 and the assembled GLM-5.2 forward pass (multi-head latent attention, data-dependent sparse attention, fine-grained MoE) are bit-exact to independent goldens at the faithful slice of Section 3; the multi-token-prediction head is verified through the `spec==greedy` invariant of Section 4; and the k-quant dequantization layer is additionally proven on 376.6M real GGUF weights. To our knowledge this ecosystem-inheritance contract has not been stated or verified by prior accelerator work.

2. **Speculative decoding proven correct in RTL** (Section 4). We compose a K-draft speculative loop into the memory-system top such that $K{+}1$ positions are verified in *one* weight load, and prove the `spec==greedy` invariant: the committed stream is a bit-exact prefix of greedy decoding under all tested accept/reject schedules. Position accuracy requires two mechanisms absent from a shared-context batch: an *intra-batch causal attention* mode (row j attends the same-pass current keys of rows $0..j{-}1$) and a die-internal per-(layer, position) KV write-back. The amortization A_{eff} is then *measured from a hardware weight-load counter*, hitting the theoretical $K{+}1$ ceiling exactly under all-accept.

3. **An adversarial, non-vacuity verification discipline** (Section 5). Every load-bearing composition gate—the speculative loop, the intra-batch oracle, each loopback family—is paired with a fault-injection build whose failure its make target asserts; the release gate pins the exact multiset of per-gate test counts (68 gates), catching silently reduced testing; and default-off features are held to structural or sequential equivalence against the pre-feature netlist where such a gate

exists (Section 3). We report a finding with, we believe, general relevance: *output-insensitive inputs defeat end-to-end equality checks*—corrupting MoE router weight codes is absorbed by top-k selection and corrupting normalization-gain LSBs is normalized away—so proving those bytes are consumed requires direct per-beat bindings on the die's inputs. A "PHY-closure loopback" applies this discipline to all five weight-input families of the die, and a case study shows the full gate catching a real cross-configuration elaboration regression that feature-local gates missed.

4. **A memory-bandwidth-bound residency methodology with measured inputs** (Section 6). The decode roofline is parameterized by *measured* expert-union growth $U(K)$ and MTP accept rates, and the stall mechanism it presumes is validated on real RTL cycles ($\sim$73$\times$ exposed stall reduction under residency), while all absolute token rates remain tagged estimates.
5. **Physical realizability evidence** (Section 6). The product top synthesizes at 87.5% LUT utilization on a commodity Kintex UltraScale+ FPGA and a bring-up harness of it places and routes (hold met, routed $F_{\max}$ 46.5 MHz); the full 753B-shape hierarchy elaborates cleanly; and a retail-parts residency prototype (1280-bit LPDDR5X, 480 GB) is supported by an analytical board routability study.

2 Background

GGUF k-quants. `llama.cpp` and its `ggml` tensor library [7] are the de facto substrate for local inference, and GGUF is their model format. The k-quant family stores weights in 256-element super-blocks with a compact hierarchy of scales: Q4_K packs 256 four-bit weights with six-bit sub-block scales and minima into 144 bytes; Q6_K uses 210 bytes; Q8_0 stores 32-element blocks of eight-bit weights with an FP16 scale. A "dynamic" mix such as UD-Q4_K_XL keeps most tensors at Q4_K while promoting sensitive tensors to higher precision. Matching this format exactly—not a nearby integer quantization—is what lets an accelerator consume the same file the community already trusts.

Fine-grained MoE, MLA, and MTP. GLM-5.2 [8] is a fine-grained MoE model [2]: each token routes to a top-k subset of 256 experts (plus one shared expert), so the *set* of weights touched per token is data-dependent. It uses multi-head latent attention (MLA) [5], which compresses the KV cache into a low-rank latent, a data-dependent sparse attention (DSA) index that selects top-k keys per query, and a multi-token-prediction (MTP) head [6] usable as a self-speculative draft in the sense of speculative decoding [3, 4].

Memory-bound roofline. For single-stream decoding of a model whose active weights exceed cache, the classic roofline model [1] reduces to a bandwidth bound: $\text{tok/s} = \text{BW} \cdot A_{\text{eff}}/B_{\text{tok}}$, where BW is sustained weight bandwidth, B_{tok} is bytes read per model pass, and A_{eff} is the mean number of committed tokens per pass (the speculative amortization). The design problem is to make BW large and $B_{\text{tok}}/A_{\text{eff}}$ small; the compute must merely keep up.

3 Accelerator Design

Bit-exact datapath. The core is a Q4_K general matrix-multiply (GEMM) unit that dequantizes super-blocks to FP32 and accumulates in a documented fixed order, followed by conversion to bf16. Mixed-type columns (Q6_K, Q8_0, F16) are handled by per-column type routing in the same

GEMM, fed by a weight loader that carries a per-tile type descriptor. The surrounding operators—MLA attention with DSA key selection, the SwiGLU expert feed-forward, the sigmoid-gated top-k MoE router, RMSNorm, interleaved RoPE, and softmax—are assembled into a full `glm_model_q4k` forward pass (embedding → $L\times$(MLA+DSA+MoE) → final norm → LM head → argmax). The assembled pass is bit-exact, on logits, argmax, and hidden state, to a numpy reference that imports the same k-quant dequantization used for the real-bytes cross-check of Section 5.

Slice and full shape. Functional verification runs at a small-but-faithful *slice* that preserves every operator and ratio (model dimension 128; six layers, three dense and three MoE; four heads; eight experts top-2 plus one shared; vocabulary 256). Three further pieces of evidence support the slice-to-full argument: (i) the real-dimension operator sweep, in which each operator is re-verified with its dominant dimension at the true 753B-configuration magnitude, under the same check contract as at the slice (the GEMM bit-exact against the `ggml` reference at reduction dimension 6144; the router's top-k and renormalization invariants at 256 experts top-8; SwiGLU, softmax, RMSNorm, and RoPE against their high-precision goldens at intermediate 2048, length 2048, dimension 6144, and rotary dimension 64 respectively—the GEMV reduction depth of the router and SwiGLU stays at slice scale, that mechanism being covered by the GEMM point); (ii) batched-shape gates in which a $\mathrm{PE}_M{>}1$ batched forward is proven row-wise bit-exact to standalone single-row runs; and (iii) a clean structural elaboration of the entire model at the true shape (dimension 6144, 78 layers, 64 heads, 256 experts, vocabulary 154,880)—a type/width check with no stimulus, which we label *elaborated*, not proven.

Memory system. A single top-level module wraps the compute die with a tier-agnostic banked read crossbar, a routed-expert cache (LRU with frequency and prefetch), an MLA latent-KV pager with storage overflow, and the weight loader; a two-clock wrapper adds the host/core domain-crossing FIFOs, and a standalone, formally proven boot-load sequencer feeds the same storage path. The fabric is *byte-agnostic*—it moves addresses, slots, and IDs, never weight bytes—so the same RTL serves streaming, full-residency, and hybrid configurations selected by parameters. With residency enabled, expert refills complete as tagged crossbar reads from the DRAM tier and the storage path carries only KV spill and the one-time boot copy, an invariant checked in simulation.

Result-invariant knobs. Several parameters (a compute-resource knob, the residency mode, the adaptive speculative depth, and every loopback and self-KV mode of Sections 4 and 5) are designed to be *result-invariant* or *default-off*: enabling them must not change the emitted tokens, and leaving them off must not change the netlist. The on direction is verified for every knob by a bit-exact gate. The off direction carries a netlist-equivalence gate where one exists: a sequential-equivalence proof for the weight-ECC path (against an ECC-removed reference derived from the pre-feature commit) and for boot integrity (against the frozen pre-feature commit itself), and a structural cell-histogram equivalence for the residency, self-KV, and CDC-protocol knobs (the first two with a built-in liveness self-test). The loopback and adaptive-depth knobs are default-off with behavioral gates only (bit-exact drop-in and output-invariance); they carry no off-netlist proof. Within those stated scopes, the knobs are optimizations, not risks.

4 Speculative Decoding Proven in RTL

Speculative decoding is the only lever that reduces the dominant roofline term: verifying K draft tokens in one model pass amortizes one weight load over up to $K{+}1$ committed tokens. Hardware

treatments of speculative decoding [13, 14, 15] report throughput; our focus is *correctness*: a composed speculating accelerator is only trustworthy if its committed stream provably equals what non-speculative decoding would have produced.

The `spec==greedy` invariant. The sequencer commits the longest accepted *prefix of the model's own argmax stream*—never a raw draft token—so by construction the committed stream must be a bit-exact prefix of greedy decoding, for any draft quality and any depth schedule. We verify this end-to-end on the composed top (compute die, expert cache, KV pager, crossbars, weight loader, and the speculative sequencer in one netlist): for $K \in \{1, 2, 3\}$ under all-accept, all-reject, and mixed partial-accept schedules, every committed token and the full logit vector of its committed row equal a standalone non-speculative reference (31 checks). Two independent fault-injection builds must fail and do: one commits raw drafts instead of model argmaxes; the other writes back the KV of a *rejected* position (a phantom-KV hazard specific to partial accepts), which diverges the very next pass's attention set.

Why a shared-context batch is not enough. A naive batched verify runs $K{+}1$ rows over a *shared, already-populated* KV cache; but row j, verifying the draft at position $p{+}j$, must attend all keys at positions $0..p{+}j{-}1$, of which the last j are the *current-token keys of rows* $0..j{-}1$ *computed in the same pass*. We therefore add, behind a default-off parameter, an *intra-batch causal attention* mode to the MLA core: each row's current-key latent is injected into the attendable key set at causal index $s{+}i$ (where s is the cached-context length and i the row index), flowing through the same RMSNorm, up-projections, RoPE, and DSA selection as cached keys, with the causal mask arising intrinsically from per-row key extents. The leaf oracle proves a batched attention pass over consecutive positions bit-exact to the serial single-row chain; the system-level batched verify then proves full-logit bit-exactness at $K \in \{1, 2, 3\}$; dropping the causal mask (a row seeing its own key) fails.

Die-internal KV write-back. Position accuracy across passes requires the die to *consume KV latents it produced*. We expose the MLA current-key latent as an output, key the pager by a flat (layer, position) index, and close the loop: at `SELF_KV=1` the die appends its computed latent per (layer, position) and re-reads it through the pager on subsequent steps, proven full-logit bit-exact against an independent (layer, position)-keyed reference at one and six layers; one-off soundness checks (documented edit-and-revert procedures in the testbench headers) confirm the reference catches layer aliasing and append corruption; with `SELF_KV=0` the netlist matches the stub-fed base's cell histogram (the added latent-observation ports contribute zero cells).

A_{eff} measured from a hardware counter. The composed top carries a `weight_loads` register that increments once per $\text{PE}_M{=}K{+}1$ outer pass, and a committed-token counter. The gate cross-checks the hardware counters against the testbench's own accounting (a hidden per-row weight reload would fail), then reports A_{eff} = tokens/loads. Measured on the composed RTL: all-accept achieves exactly the ceiling ($K{+}1$ tokens per load: 2.00, 3.00, 4.00 for $K{=}1, 2, 3$), all-reject the floor (1.00), and mixed schedules the expected intermediate values. This turns the roofline's amortization term from an assumption into a measured property of the netlist; the *production* A_{eff} then depends only on the model's accept rate, an externally measured input (Section 6).

Claim	Evidence	Status
k-quant dequant vs real GGUF	bitwise-equal to llama.cpp `ggml` on 376.6M weights	proven
Q4_K GEMM core	bit-exact vs `ggml` reference (160 tests, 40 tiles)	proven
Assembled model forward	bit-exact vs assembled numpy golden (1,155 tests)	proven
Mixed-type path (Q6_K/Q8_0/F16)	bit-exact incl. 24-tile mixed sequence	proven
Speculative loop (`spec==greedy`)	committed stream $\equiv$ greedy prefix, 3 schedules $\times$ $K \leq 3$	proven
A_{eff} amortization	hardware weight-load counter; all-accept $= K+1$/load	measured
Intra-batch causal attention	batch $\equiv$ serial chain, full-logit	proven
Die-internal KV write-back	bit-exact vs independent (layer,pos) reference	proven
PHY-closure loopback (5 families)	fabric round-trip bit-exact; per-beat bindings on rw/lw/gn	proven
Memory/control plane	BMC (7 ctrl + ECC ring) + k-induction (5)	proven (formal)
Result-invariant knobs, off-direction	cell-histogram or sequential equivalence (residency, self-KV, CDC, ECC, boot)	proven
Full 753B-shape hierarchy	type/width elaboration at true dimensions	elaborated
FPGA fit (compact config)	routed on Kintex US+, hold met, $F_{\max}$ 46.5 MHz	measured
Cycle-accurate stall mechanism	per-token stall on real cycles; residency $\sim 73\times$	measured
Single-user throughput	roofline with measured $U(K)$, r, A_{eff}	estimated[EST]
End-to-end 467 GB run; silicon rate	—	not done

Table 1: The verification ledger. Statuses: *proven* = a gated bit-exact simulation or a solver proof; *measured* = real RTL cycles or a real place-and-route; *elaborated* = structural only, no functional golden; *estimated* = a roofline projection. Every "proven" and "measured" row is reproducible from the repository; the load-bearing composition gates are paired with fault-injection builds that must fail.

5 Verification Methodology

Verification is organized as a ledger. Each claim is *proven* (a gated bit-exact simulation or a solver proof), *measured* (real RTL cycles or a real place-and-route), *elaborated* (structural only), or *estimated* (a roofline). Table 1 summarizes; this section describes the load-bearing mechanisms.

Closing the self-referential gap. The assembled forward pass is bit-exact to a numpy golden, but that golden is the author's own reimplementation of `ggml`. To close the gap to the software the community runs, we parse two real published GGUF files (a Qwen2.5-0.5B Q4_K_M and a SmolLM2-135M Q8_0 distribution), extract every Q4_K and Q6_K tensor's raw super-blocks and every Q8_0 tensor's blocks, and dequantize each block two ways: with our reference, and with llama.cpp's own `dequantize_row_q4_K`/`_q6_K`/`_q8_0` compiled from the built `ggml` library. The FP32 outputs are compared bitwise (as `uint32`, not within a tolerance). All 376,586,240 weights

match. The dequantization is format-level and file-agnostic—a k-quant block's bytes decode identically whatever model they came from—so bitwise identity on these distributions' blocks transfers to any k-quant file, including the 467 GB target checkpoint. By transitivity with the RTL $\equiv$ reference gates, the hardware dequantization equals the real files' `ggml` dequantization. The scope is deliberate: the GEMM's FP32 accumulation follows *our* documented order, not llama.cpp's runtime order (which differs); whole-runtime equality is out of contract.

Injection pairing (non-vacuity). A gate that cannot fail proves nothing. Every load-bearing composition gate—the speculative loop (two injections: raw-draft commit, phantom KV), the intra-batch causal oracle (dropped mask), and each loopback family—is therefore paired with at least one fault-injection build that must make the gate fail, and the gate's make target asserts that failure on every run. This is mutation analysis [22, 23] specialized to one hand-chosen, semantically meaningful mutant per claim, asserted alongside the gate itself rather than as an offline qualification campaign; the remaining unit gates are self-checking testbenches whose counts the manifest below pins.

Exact-count manifests. Pass banners of the form "all N tests passed" are conventionally grepped with a wildcard N, which lets a testbench silently run *fewer* tests (a dropped compile flag, a reduced loop) while CI stays green. Our release gate pins the exact multiset of test counts for all 68 gates in a manifest; any drift—fewer tests, a vanished configuration, an extra silent run—fails the gate. In effect the test count itself becomes a verified contract.

Output-insensitive bytes and direct bindings. While extending the loopback proof (below) to the MoE router and normalization-gain inputs, we found that end-to-end token equality *cannot* bind them: corrupting a router weight code by a full quantization step is absorbed by top-k expert selection (the ranking rarely changes), and corrupting a normalization gain's LSB is normalized away downstream. A verification flow that relied only on committed-token equality would silently accept a die that reads garbage on those inputs. The fix is a *direct per-beat binding*: hierarchical probes assert that the die's presented input equals the expected byte for the live request key on every beat. With the binding in place, the same corruptions that end-to-end equality misses are caught immediately. We believe this observation—an instance of the classic observability problem [24] surfacing at the integration level—is a practical hazard for any accelerator verified primarily by end-to-end token comparison.

PHY-closure loopback. Pre-silicon integration commonly stubs the die's weight inputs from the testbench, leaving a gap: the memory fabric is verified to *count* the right transactions while the die never actually consumes fabric-returned bytes. We close this for *all five* weight-input families of the die (attention codes, FFN expert codes, router codes, LM-head columns, normalization gains): behind default-off parameters, each family's bytes are routed *out* of the die as tagged reads through the real banked crossbar and back *in*, clock-gating the die per beat, and the committed stream is proven bit-exact against the stub-fed reference (thousands of round-trips per decode; per-family non-vacuity counters). Corruption-injection builds are asserted for the attention, expert, gain, and router families—the latter two caught by the direct per-beat bindings that token equality cannot see; the LM-head family is bound by the same per-beat check. What remains external is the analog PHY itself (vendor IP); the digital consume-path is closed.

Formal verification of the control plane. The memory-system controllers—the banked DRAM crossbar, the storage-read fabric, the boot loader, the KV pager, the speculative-decode sequencer, the clock-throttle controller, and the expert cache in both prefetch modes—are checked by bounded model checking (with an ECC-datapath variant), and five controllers are lifted to unbounded k-induction, with no counterexample from reset. No formal proof touches the numeric datapath; that plane is held by the bit-exact goldens. Residual bounded gaps are documented rather than hidden.

Case study: the full gate catches what feature gates miss. During the intra-batch causal work, an unconditional wire computed a zero-padding width as $(\mathrm{IDXW}+1)-\mathrm{DRW}$; in an unrelated top instantiating the same MLA core with a small context and a large batch, the replication count went negative and elaboration failed—while every gate of the *feature* passed, because the feature's own configurations never entered that corner. The full release gate (all 68 gates, all tops) caught it. We take this as evidence for running whole-project gates on every RTL change: in a parameterized codebase, a shared leaf can break a sibling top at parameters the feature author never exercises.

6 Design-Space Analysis and Evaluation

Measured roofline inputs. The bytes-per-pass denominator has two parts that speculation amortizes differently. The routed-expert stream is ≈14 GB per pass, and under a speculative chain the *union* of experts touched across the verified positions grows by a factor $U(K)$; the resident hot set (attention projections, dense and shared FFN, router, LM head) is ≈11 GB, read once per pass. We measure $U(K)$ directly by tracing exact expert selections on a same-family model (GLM-4.5-Air): $U(2)\approx1.6$, $U(4)\approx2.65$, $U(8)\approx4.3$; concurrent systems work reports compatible union growth [21]. We also measure the MTP accept-rate profile r—first-position $r_1\approx0.87$ with steep per-position decay—which places the memory-bound optimum at shallow depth (K=1–2) and yields a production $A_{\mathrm{eff}}\approx1.9$ at K=1; the RTL-measured mechanism of Section 4 guarantees the amortization is realized, and the adaptive depth controller is proven output-invariant under any depth schedule (its own gate) and designed to settle at the measured optimum.

Cycle-accurate validation of the mechanism. A cycle-accurate testbench drives the assembled system with a storage read-latency model engaged, holding every committed token bit-exact against a standalone reference while counting cycles. On the verified slice the compute-die baseline is ≈10,896 cycles/token. Exposed storage stall is linear in read latency and proportional to miss count—the memory-bound assumption, now *measured*—and the residency configuration makes stall independent of storage latency: at a 1024-cycle latency, streaming exposes 2,567 stall cycles/token whereas residency exposes 35, a ∼73× reduction. This validates the residency pivot on real cycles even though the absolute product token rate remains a projection.

FPGA fit. The Q4_K product top, in a compact configuration, synthesizes on a Kintex UltraScale+ XCKU3P at 142,320 LUTs (87.5% utilization), ∼100K flip-flops, 421 DSPs, and zero BRAM; a bring-up harness of the same top (the wide memory-side ports terminated on-chip so the fit is not I/O-limited) then places and routes at ∼141.3K LUTs with hold met and a routed maximum frequency of 46.5 MHz, reached through bit-exactness-preserving repipelining rounds (a campaign that raised routed $F_{\max}$ 4.6×). This establishes that the datapath is physically realizable on commodity silicon; it is a compact slice, not the full model at product speed, and we do not extrapolate a token rate from it.

Projected throughput. Combining a resident LPDDR5X pool with the measured $U(K)$, r, and the RTL-measured A_{eff} mechanism, the roofline projects a residency design point of order 80–110 tok/s[EST] for the single-user box—presuming a MAC array sized to consume that bandwidth (∼9.4K dequant lanes at 490 MHz for a 1.1 TB/s pool, ∼12.7K for 1.54 TB/s, with attention and MoE holding dedicated engines that each absorb the full bandwidth in their own sequential phase). Three cautions make this an *optimistic ceiling*: quoted bandwidths are peak, not sustained (∼70–85% is typical); measured lane scaling is sublinear (4× lanes yielded 2.40× on the slice), so an under-provisioned array, not memory, becomes the bound; and no board exists. For context, the literature reports roughly 1.3–13 tok/s for GPU/flash offloading of models of this scale [16, 17], and 512 GB-class unified-memory workstations are community-reported around 15–25 tok/s; our figures are projections with measured inputs, not silicon measurements.

Prototype design point and routability. A physically buildable prototype uses commodity retail parts: twenty 24 GB LPDDR5X packages on a 1280-bit board-level bus give 480 GB resident at ≈1.54 TB/s. Because the wide bus leaves the package, we perform a first-principles routability study. An escape-routing model (via-in-pad HDI, the ring-cut channel-crossing bound) estimates the ∼2,800-signal escape of the host BGA needs on the order of four signal layers, within the six a 12-layer stack provides; a placement check confirms the twenty packages and the host fit within a 130×110 mm board with courtyard and edge clearance. This is analysis, not a routed board.

7 Related Work

GGUF/k-quant hardware. SECDA-LLM [9] integrates an FPGA accelerator for `ggml` k-quant matrix multiplies into llama.cpp on a PYNQ-Z1, and F-BFQ [10] accelerates GGUF block quantization (Q2_K/Q3_K) with dynamic format switching on a Kria KV260; both target TinyLlama/GPT-2-class models, offload individual kernels, and state no bitwise contract with `ggml`. Edge-FPGA LLM accelerators more broadly (LlamaF [11], TeLLMe [12], and full-model efforts such as llama-fpga) use custom, ternary, or AWQ-style quantization rather than the GGUF byte format. Our delta is not the genre but the *contract* (bitwise inheritance of the real checkpoint bytes, proven at 376.6M-weight scale) and the *scope* (a full MLA+DSA+MoE+MTP datapath elaborated at 753B shape; we found no prior RTL for DSA or MTP).

Speculative decoding in hardware. HADES [13] adds hardware-level speculative support to an LLM accelerator; SpecPIM [14] co-designs speculative inference for processing-in-memory; SpecMamba [15] brings speculative decoding to FPGA for state-space models. These report performance; none, to our knowledge, states or proves a committed-stream-equals-greedy invariant in RTL, measures the amortization from a hardware counter, or identifies the intra-batch causal attention and KV write-back mechanisms required for position-accurate batched verification in a latent-attention model.

MoE offload and appliances. Software systems stream experts across tiers (LLM-in-a-flash [16], MoE-Infinity [17]); architecture proposals add MoE-aware memory hierarchies (Duplex [18], Cambricon-LLM [19], Stratum [20]). These pursue throughput on shared or datacenter silicon; our design targets the single-user offline residency point, and its contribution is the verified RTL artifact and methodology rather than a new hierarchy proposal. Concurrent measurements of expert-union growth under multi-token verification [21] corroborate our $U(K)$ inputs.

Verification methodology. Mutation analysis and functional qualification (Certitude-style; MCY [23] in the open-source flow) established must-fail fault builds; observability-based coverage [24] established that activation without propagation is invisible to output checks. Our discipline specializes both to a per-claim, always-on CI form (one semantically chosen mutant per gate; exact-count manifests), and documents a concrete integration-level instance—output-insensitive weight families in an MoE model—where end-to-end token equality is provably insufficient and per-beat input bindings are required.

8 Limitations and Honest Scope

We restate the boundaries plainly. (1) There is no fabricated silicon and no running board; the remaining physical gates are vendor PHY IP, compiled SRAM macros with their BIST collars, DFT insertion, and tapeout. (2) All token rates are roofline projections tagged [EST]; the FPGA result is a compact place-and-route, not the full model at speed; measured lane scaling is sublinear, so the projections are optimistic ceilings. (3) Functional proofs run at a faithful slice; the true 753B shape is structurally elaborated only (a full-scale functional simulation is computationally infeasible), with real-dimension operator sweeps as the bridge. (4) The numeric contract is `ggml`-exact dequantization plus a documented accumulation order; whole-runtime equality with llama.cpp is deliberately out of contract, and the 467 GB checkpoint has not been run end-to-end. (5) The MTP accept rate is measured on a same-family proxy (GLM-4.5-Air), not on GLM-5.2 itself; one architecture parameter (`kv_lora`=512) follows the DeepSeek convention pending direct confirmation. (6) The board study is analytical, not a routed design. We regard stating these boundaries as part of the contribution: for a system an auditor must sign off on, an honest ledger is the argument.

9 Conclusion

We have described a verification-first RTL accelerator whose defining property is a checkable contract: the hardware dequantizes real GGUF checkpoint bytes exactly as the community's own software does, and therefore inherits the k-quant ecosystem by construction. On top of that contract we proved a speculative decoding loop correct in RTL (committed stream $\equiv$ greedy prefix, amortization measured from a hardware counter), formalized an adversarial verification discipline whose gates must be able to fail—and showed why end-to-end equality cannot see output-insensitive bytes—and grounded a memory-bandwidth-bound residency design in measured inputs and real-cycle stall validation. The datapath places and routes on commodity FPGA silicon. What remains between this artifact and a device is named, not hidden: PHY IP, macros, DFT insertion, a board, a tapeout. The full engineering record, and every gate that backs the claims above, is open.

Reproducibility

Every "proven" and "measured" row of Table 1 is reproducible from the public repository at `https://github.com/Wick-Lim/AIPU` via the corresponding build targets: the bit-exact simulation gates and their paired injection builds, the GGUF cross-check tool, the exact-count release manifest (`make release-gate-strict`), the speculative and loopback gates, the formal model-checking targets, and the cycle-accurate performance harness.

Acknowledgments

The GGUF cross-check builds on the open-source `ggml`/llama.cpp project and the published GLM open weights; this work is an independent inference accelerator for those models.

References

[1] S. Williams, A. Waterman, and D. Patterson. Roofline: an insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009.

[2] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean. Outrageously large neural networks: the sparsely-gated mixture-of-experts layer. In *ICLR*, 2017. arXiv:1701.06538.

[3] Y. Leviathan, M. Kalman, and Y. Matias. Fast inference from transformers via speculative decoding. In *ICML*, 2023. arXiv:2211.17192.

[4] C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, and J. Jumper. Accelerating large language model decoding with speculative sampling. arXiv:2302.01318, 2023.

[5] DeepSeek-AI. DeepSeek-V2: a strong, economical, and efficient mixture-of-experts language model. arXiv:2405.04434, 2024.

[6] DeepSeek-AI. DeepSeek-V3 technical report. arXiv:2412.19437, 2024.

[7] G. Gerganov and the llama.cpp/ggml contributors. llama.cpp and the ggml tensor library. Software, `https://github.com/ggml-org/llama.cpp`.

[8] Zhipu AI / THUDM. GLM-5.2 open weights (GGUF k-quant distribution). Model weights, `https://huggingface.co/unsloth/GLM-5.2-GGUF`.

[9] J. Haris, R. Saha, W. Hu, and J. Cano. Designing efficient LLM accelerators for edge devices (SECDA-LLM). arXiv:2408.00462, 2024.

[10] J. Haris and J. Cano. F-BFQ: flexible block floating-point quantization accelerator for LLMs. arXiv:2510.13401, 2025.

[11] H. Xu, Y. Li, and S. Ji. LlamaF: an efficient Llama2 architecture accelerator on embedded FPGAs. arXiv:2409.11424, 2024.

[12] Y. Qiao, Z. Chen, Y. Zhang, Y. Wang, and S. Huang. TeLLMe v2: an efficient end-to-end ternary LLM prefill and decode accelerator with table-lookup matmul on edge FPGAs. arXiv:2510.15926, 2025.

[13] Z. Yang, Y. Jin, and X. Xu. HADES: hardware accelerated decoding for efficient speculation in large language models. arXiv:2412.19925, 2024.

[14] C. Li et al. SpecPIM: accelerating speculative inference on PIM-enabled systems via architecture-dataflow co-exploration. In *ASPLOS*, 2024.

[15] L. Zhong et al. SpecMamba: accelerating Mamba inference on FPGA with speculative decoding. arXiv:2509.19873, 2025.

[16] K. Alizadeh et al. LLM in a flash: efficient large language model inference with limited memory. arXiv:2312.11514, 2023.

[17] L. Xue et al. MoE-Infinity: efficient MoE inference on personal machines with sparsity-aware expert cache. arXiv:2401.14361, 2024.

[18] S. Yun et al. Duplex: a device for large language models with mixture of experts, grouped query attention, and continuous batching. In *MICRO*, 2024.

[19] Z. Yu et al. Cambricon-LLM: a chiplet-based hybrid architecture for on-device inference of 70B LLM. In *MICRO*, 2024. arXiv:2409.15654.

[20] Y. Pan et al. Stratum: system-hardware co-design with tiered monolithic 3D-stackable DRAM for efficient MoE serving. In *MICRO*, 2025. arXiv:2510.05245.

[21] A. Saxena, P.-A. Tsai, H. Taneja, A. Jaleel, and M. Qureshi. Utility-driven speculative decoding for mixture-of-experts. arXiv:2506.20675, 2025.

[22] Y. Jia and M. Harman. An analysis and survey of the development of mutation testing. *IEEE Transactions on Software Engineering*, 37(5):649–678, 2011.

[23] YosysHQ. MCY: mutation cover with Yosys. Software, `https://github.com/YosysHQ/mcy`.

[24] F. Fallah, S. Devadas, and K. Keutzer. OCCOM: efficient computation of observability-based code coverage metrics for functional verification. In *DAC*, 1998.